

1)AbsoluteDifference

**You are given a function,
int findCount(int arr[], int length, int num, int diff);**

The function accepts an integer array 'arr', its length and two integer variables 'num' and 'diff'. Implement this function to find and return the number of elements of 'arr' having an absolute difference of less than or equal to 'diff' with 'num'.

Note: In case there is no element in 'arr' whose absolute difference with 'num' is less than or equal to 'diff', return -1.

Example:

Input:

arr: 12 3 14 56 77 13

num: 13

diff: 2

Output:

3

Explanation:

Elements of 'arr' having absolute difference of less than or equal to 'diff' i.e. 2 with 'num' i.e. 13 are 12, 13 and 14.

```

1- import java.util.Scanner;
2- public class AbsoluteDifference {
3
4-     public static int AbsDifference(int arr[], int length, int num, int diff){
5
6         int count = 0;
7         for(int i = 0; i < length; i++){
8             if(Math.abs(num - arr[i]) <= 2){
9                 count++;
10            }
11        }
12
13        return count > 0 ? count : -1;
14    }
15
16-     public static void main(String[] args) {
17         Scanner sc = new Scanner(System.in);
18
19         System.out.println("Enter size of arr: ");
20         int n = sc.nextInt ();
21
22         int arr[] = new int[n];
23         System.out.println("Enter elements: ");
24         for (int i = 0; i < n; i++){
25             arr[i] = sc.nextInt ();
26         }
27
28         System.out.println("Enter num: ");
29         int num = sc.nextInt ();
30
31         System.out.println("Enter difference: ");
32         int diff = sc.nextInt ();
33
34         System.out.println ("Absolute Diff count: " + AbsDifference(arr, n, num, diff));
35     }
36 }

```

2)Anagram

You are given two strings, s and t . Your task is to determine if it's possible to rearrange the characters of s to form the string t . Write a function that returns True it's possible to create t by rearranging the characters of s and False otherwise.

Input

Two strings, s and t where the length of s and t are between 1 and 1000 characters.

Output

Return True if it's possible to create t by rearranging the characters of s and False otherwise

Example:

s="listen"

t="silent"

Output:

True

```

1 import java.util.*;
2
3 public class Anagram{
4
5     public static boolean CheckAnagram(String s, String t){
6
7         s = s.toLowerCase();
8         t = t.toLowerCase();
9
10        char []arr1 = s.toCharArray();
11        char []arr2 = t.toCharArray();
12
13        Arrays.sort(arr1); // e,i,l,n,s,t
14        Arrays.sort(arr2); // e,i,l,n,s,t
15
16        if(Arrays.equals(arr1,arr2)){
17            return true;
18        }else{
19            return false;
20        }
21    }
22
23    public static void main(String[] args) {
24        Scanner sc = new Scanner(System.in);
25
26        System.out.println("Enter first string: ");
27        String s=sc.nextLine(); // Silent
28
29        System.out.println("Enter second string: ");
30        String t=sc.nextLine(); // listen
31
32        System.out.println("Output: " + CheckAnagram(s, t));
33    }
34 }

```

3) Autobiographical Number

Problem Statement :

An Autobiographical Number is a number N such that the first digit of N represents the count of how many zeroes are there in N, the second digit represents the count of how many ones are there in N and so on.

You are given a function, def FindAutoCount(n):

The function accepts string “n” which is a number and checks whether the number is an autobiographical number or not. If it is, an integer is returned, i.e. the count of distinct numbers in ‘n’. If not, it returns 0.

Assumption:

The input string will not be longer than 10 characters.

Input string will consist of numeric characters.

Note:

If string is None return 0.

Example:

Input:

n: “1210”

Output:

3

Explanation:

0th position in the input contains the number of 0 present in input, i.e. 1, in 1st position the count of number of 1s in input i.e. 2, in 2nd position the count of 2s in input i.e. 1, and in 3rd position the count of 3s i.e. 0, so the number is an autobiographical number.

Now unique numbers in the input are 0, 1, 2, so the count of unique numbers is 3. So 3 is returned.

```
1  #include <iostream>
2  #include<bits/stdc++.h>
3  using namespace std;
4  int FinndAutoCount (string n)
5  {
6      int sum = 0;
7      set<char> st;
8      for (int i = 0; i < n.size (); i++){
9          sum += (n[i] - '0');    // converting the string character into integer value and
                                  adding it to sum
10         st.insert (n[i]);
11     }
12
13     if (sum != n.size ())
14         return 0;
15
16     return st.size ();
17 }
18
19 int main () {
20     string n;
21
22     cout<<"Enter no: "<<endl;
23     cin >> n;
24
25     cout <<"Result: "<< FinndAutoCount (n);
26     return 0;
27 }
```

4) BinaryOperations

Problem Description :

The Binary number system only uses two digits, 0 and 1 and number system can be called binary string. You are required to implement the following function:

```
int OperationsBinaryString(char* str);
```

The function accepts a string str as its argument. The string str consists of binary digits eparated with an alphabet as follows:

- A denotes AND operation
- B denotes OR operation
- C denotes XOR Operation

You are required to calculate the result of the string str, scanning the string to right taking one opearation at a time, and return the same.

Note:

- No order of priorities of operations is required
- Length of str is odd
- If str is NULL or None (in case of Python), return -1

Input:

str: 1C0C1C1A0B1

Output:

1

Explanation:

The alphabets in str when expanded becomes “1 XOR 0 XOR 1 XOR 1 AND 0 OR 1”, result of the expression becomes 1, hence 1 is returned.

Sample Input:

0C1A1B1C1C1B0A0

Output:

0

```
1 import java.util.*;
2 public class BinaryOperations {
3
4     public static int OperationsBinaryString(String str){
5         if(str == null) return 0;
6
7         int result = str.charAt(0) - '0';
8
9         for(int i = 1; i < str.length(); i += 2){
10             char operation = str.charAt(i);
11             int nextDigit = str.charAt(i+1) - '0';
12
13             switch(operation){
14                 case 'A':
15                     result = result & nextDigit;
16                     break;
17                 case 'B':
18                     result = result | nextDigit;
19                     break;
20                 case 'C':
21                     result = result ^ nextDigit;
22                     break;
23                 default:
24                     return -1;
25             }
26         }
27         return result;
28     }
29
30     public static void main(String[] args) {
31         Scanner sc = new Scanner(System.in);
32         String str = sc.next();
33         System.out.println(OperationsBinaryString(str));
34         sc.close();
35     }
36 }
```

5) Binary to dec

Problem Statement

Convert binary no to decimal

Input: 1010

Output: 10

```
1 import java.util.*;
2
3 public class BinaryToDec{
4     public static int decTobin(int n) {
5         if (n == 0) {
6             return 0;
7         }
8
9         int ans = 0;
10        int base = 1; // This represents the place value (1, 10, 100, ...)
11
12        while (n > 0) {
13            int lastBit = n % 10; // Get the last bit (either 0 or 1)
14            ans += lastBit * base; // Add it to the current base value
15            base *= 2; // Move to the next place value (1 -> 10 -> 100, etc.)
16            n /= 10; // Right shift n by 1 (equivalent to n = n / 2)
17        }
18
19        return ans;
20    }
21
22    public static void main(String[] args) {
23        Scanner sc = new Scanner(System.in);
24
25        System.out.println("Enter the the decimal number: ");
26        int n = sc.nextInt();
27
28        System.out.println("Output: "+ decTobin(n));
29    }
30 }
```

6)Bulb Switch

Problem Statement :

N light bulbs are connected by a wire. Each bulb has a switch associated with it, however due to faulty wiring, a switch also changes the state of all the bulbs to the right of current bulb. Given an initial state of all bulbs, Find the minimum number of switches you have to press to turn on all the bulbs. You can press the same switch multiple times.

Note: 0 represents the bulb is off and 1 represents the bulb is on.

Note: $A[] = [0\ 1\ 0\ 1]$

Output: 4

Explanation:

Press switch 0: $[1\ 0\ 1\ 0]$ //1

Press switch 1: $[1\ 1\ 0\ 1]$ //2

Press switch 2: $[1\ 1\ 1\ 0]$ //3

Press switch 3: $[1\ 1\ 1\ 1]$ //4

Input: $A[] = [1\ 0\ 0\ 0\ 0]$

Output: 1

```

1 import java.util.*;
2 public class BulbSwitch
3 {
4     public static int BulbSwitch(int arr[], int n)
5     {
6         int count = 0;
7         for(int i = 0; i < n; i++){
8             if(arr[i] == 0){
9                 arr[i] = 1;
10                for(int j = i+1; j < n; j++){
11                    if(arr[j] == 1){
12                        arr[j] = 0;
13                    }else{
14                        arr[j] = 1;
15                    }
16                }
17                count++;
18            }
19        }
20        return count;
21    }
22
23    public static void main(String[] args) {
24        Scanner sc = new Scanner(System.in);
25
26        int arr[] = {0,1,0,1};
27        int n = arr.length;
28
29        System.out.println("Minimum no of switches to press: " + BulbSwitch(arr, n));
30    }
31 }

```

7) Chocolate Distribution

Given an array of N integers where each value represents the number of chocolates in a packet. Each packet can have a variable number of chocolates. There are m students, the task is to distribute chocolate packets such that:

Each student gets one packet.

The difference between the number of chocolates in the packet with maximum chocolates and the packet with minimum chocolates given to the students is minimum.

Examples:

Input : arr[] = {7, 3, 2, 4, 9, 12, 56} , m = 3

Output: Minimum Difference is 2

Explanation:

We have seven packets of chocolates and we need to pick three packets for 3 students

If we pick 2, 3 and 4, we get the minimum difference between maximum and minimum packet sizes.

Input : arr[] = {3, 4, 1, 9, 56, 7, 9, 12} , m = 5

Output: Minimum Difference is 6

Input : arr[] = {12, 4, 7, 9, 2, 23, 25, 41, 30, 40, 28, 42, 30, 44, 48, 43, 50} , m = 7

Output: Minimum Difference is 10

```

1 import java.util.*;
2 public class ChocolateDistribution {
3
4     public static int chocolateDistribution (int arr[], int m)
5     {
6         if(arr.length == 0 || m == 0){
7             return 0;
8         }
9
10        Arrays.sort(arr);
11
12        if(m > arr.length-1){
13            return -1;
14        }
15
16        int minDiff = Integer.MAX_VALUE;
17
18        for(int i = 0; i < arr.length; i++){
19            int j = i + m - 1;
20
21            if(j >= arr.length){
22                break;
23            }
24
25            int diff = arr[j] - arr[i];
26            minDiff = Math.min(minDiff, diff);
27        }
28        return minDiff;
29    }
30 }
31
32 public static void main(String[] args) {
33     int arr[] = {12, 4, 7, 9, 2, 23, 25, 41, 30, 40, 28, 42, 30, 44, 48, 43, 50};
34     int m = 7;
35
36     int result = chocolateDistribution(arr, m);
37
38     if (result != -1) {
39         System.out.println("Minimum difference is " + result);
40     } else {
41         System.out.println("Invalid input");
42     }

```

8) CountCarry

Problem Statement

A carry is a digit that is transferred to left if sum of digits exceeds 9 while adding two numbers from right-to-left one digit at a time

You are required to implement the following function.

Int NumberOfCarries(int num1 , int num2);

The functions accepts two numbers 'num1' and 'num2' as its arguments. You are required to calculate and return the total number of carries generated while adding digits of two numbers 'num1' and ' num2'.

Assumption: num1, num2>=0

Example:

Input

Num 1: 451

Num 2: 349

Output

2

Explanation:

Adding 'num 1' and 'num 2' right-to-left results in 2 carries since (1+9) is 10. 1 is carried and (5+4=1) is 10, again 1 is carried. Hence 2 is returned.

Sample Input

Num 1: 23

Num 2: 563

Sample Output

0

```
1 #include <iostream>
2 using namespace std;
3 int CountCarry(int no1, int no2){
4
5     int carry = 0, count = 0, sum = 0;
6
7     while (no1 != 0 || no2 != 0) {
8         int val1 = no1 % 10;
9         int val2 = no2 % 10;
10
11         sum = carry + val1 + val2;
12
13         if (sum > 9) {
14             carry = 1;
15             count++;
16         } else {
17             carry = 0;
18         }
19
20         no1 = no1 / 10;
21         no2 = no2 / 10;
22     }
23     return count;
24 }
25
26 int main()
27 {
28     int x, y;
29     int result;
30
31     cout<<"Enter first no: "<<endl;
32     cin >> x;
33
34     cout<<"Enter second no: "<<endl;
35     cin >> y;
36
37     result = CountCarry(x, y);
38     cout <<"Result: "<<result;
39
40     return 0;
41 }
```

9) Dec to Bin

Problem Statement

Convert decimal no to binary

Input: 10

Output: 1010

```
1 import java.util.*;
2
3 public class DecToBinary{
4     public static String decbin(int n){
5         if(n == 0){
6             return "0";
7         }
8
9         String ans = "";
10        while (n > 0) {
11            if ((n & 1) == 1) {
12                ans = "1" + ans;
13            } else {
14                ans = "0" + ans;
15            }
16            n = n >> 1;
17        }
18        return ans;
19    }
20    public static int decTobin(int n) {
21        if (n == 0) {
22            return 0;
23        }
24
25        int ans = 0;
26        int place = 1; // This represents the place value (1, 10, 100, ...)
27        while (n > 0) {
28            int lastBit = n & 1; // Get the last bit (either 0 or 1)
29            ans += lastBit * place; // Add it to the current place value
30            place *= 10; // Move to the next place value (1 -> 10 -> 100, etc.)
31            n = n >> 1; // Right shift n by 1 (equivalent to n = n / 2)
32        }
33        return ans;
34    }
35    public static void main(String[] args) {
36        Scanner sc = new Scanner(System.in);
37        System.out.println("Enter the the decimal number: ");
38        int n = sc.nextInt();
39        System.out.println("Output: "+ decTobin(n));
40    }
41 }
```

10) Write a program in Java such that it takes a lower limit and upper limit as inputs and print all the intermediate palindrome numbers.

Test Cases:

TestCase 1:

Input :

10 , 80

Expected Result:

11 , 22 , 33 , 44 , 55 , 66 , 77.

Test Case 2:

Input:

100,200

Expected Result:

101 , 111 , 121 , 131 , 141 , 151 , 161 , 171 , 181 , 191.

```

1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.Scanner;
4 public class DisplayPalindrome {
5
6     public static int palindrome (int no1)
7     {
8         int rem = 0;
9         int div = no1;
10        while (div != 0)
11        {
12            int r = div % 10;
13            rem = (rem * 10) + r;
14            div = div / 10;
15        }
16        return rem;
17    }
18
19    public static void main(String[] args) {
20        Scanner sc = new Scanner (System.in);
21
22        System.out.println ("Enter Upper and Lower Limit");
23        int ul = sc.nextInt ();
24        int ll = sc.nextInt ();
25
26        for (int i = ul; i <= ll; i++){
27            if (i == palindrome (i))
28                System.out.print(i+" ");
29        }
30    }
31 }

```

11) The program is supposed to calculate the sum of distance between three points from each other.

For

$x_1 = 1$ $y_1 = 1$

$x_2 = 2$ $y_2 = 4$

$x_3 = 3$ $y_3 = 6$

Distance is calculated as : $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$

```
1 import math
2
3 def calculate_distance(x1, y1, x2, y2):
4     return math.sqrt(math.pow(x2 - x1, 2) + math.pow(y2 - y1, 2))
5
6 def total_distance(x1, y1, x2, y2, x3, y3):
7     first_diff = calculate_distance(x1, y1, x2, y2)
8     second_diff = calculate_distance(x2, y2, x3, y3)
9     third_diff = calculate_distance(x1, y1, x3, y3)
10
11     return round(first_diff, 2), round(second_diff, 2), round(third_diff, 2)
12
13 def main():
14     x1, y1 = map(float, input("Enter coordinates for point 1 (x1 y1): ").split())
15     x2, y2 = map(float, input("Enter coordinates for point 2 (x2 y2): ").split())
16     x3, y3 = map(float, input("Enter coordinates for point 3 (x3 y3): ").split())
17
18     first_diff, second_diff, third_diff = total_distance(x1, y1, x2, y2, x3, y3)
19     print("Distances:")
20     print(f"Between point 1 and point 2: {first_diff}")
21     print(f"Between point 2 and point 3: {second_diff}")
22     print(f"Between point 1 and point 3: {third_diff}")
23
24 if __name__ == "__main__":
25     main()
```

12) Given an array, find the number of occurrences of each element in the array.

Sample Test Case 2:

Input:

arr[] = {10,5,10,15,10,5}

Output:

10 - 3

5 - 2

15 - 1

```

1 import java.util.HashMap;
2
3 public class ElementOccurrences {
4     public static HashMap<Integer, Integer> Occurrence(int[] array) {
5         HashMap<Integer, Integer> occu = new HashMap<>();
6
7         for(int elem : array){
8             if(occu.containsKey(elem)){
9                 occu.put(elem, occu.get(elem) + 1);
10            }
11            else{
12                occu.put(elem, 1);
13            }
14        }
15        return occu;
16    }
17
18    public static void main(String[] args) {
19        int arr[] = {10, 5, 10, 15, 10, 5};
20        HashMap<Integer, Integer> result = Occurrence(arr);
21
22        for (int key : result.keySet()) {
23            System.out.println(key + " - " + result.get(key));
24        }
25    }
26 }

```

13) Elevation Point

Problem Statement :

Input : N = 7, arr = [1,2,3,4,3,2,1]

Output: 4

Explanation: 4 is the elevation point

Input: N = 2, arr = [5,3]

Output: 5

```

1- import java.util.*;
2  public class ElevationPoint
3  {
4      public static int elevation(int arr[], int n) {
5          if(n == 1) return arr[0];
6
7          if(arr[0] > arr[1]){
8              return arr[0];
9          }
10         if(arr[n-1] > arr[n-2]){
11             return arr[n-1];
12         }
13         int low = 1, high = n-2;
14         while(low <= high){
15             int mid = low + (high - low) / 2;
16             if((arr[mid] > arr[mid-1]) && (arr[mid] > arr[mid+1])){
17                 return arr[mid];
18             }
19             else if(arr[mid-1] < arr[mid]){
20                 low = mid + 1;
21             }
22             else{
23                 high = mid - 1;
24             }
25         }
26         return -1;
27     }
28     public static void main(String[] args) {
29         Scanner sc = new Scanner(System.in);
30
31         System.out.println("Enter no of elements in array: ");
32         int n = sc.nextInt();
33
34         System.out.println("Enter the elements of array: ");
35         int arr[] = new int[n];
36         for(int i = 0; i < n; i++){
37             arr[i] = sc.nextInt();
38         }
39
40         System.out.println("Elevation point is: " + elevation(arr, n));
41     }
42 }

```

14) Encode Number

Encode the Number:

You work in the message encoding department of a national security message agency. Every message that is sent from or received in your office is encoded. You have an integer N and each digit of N is squared

and the squares are concatenated together to encode the original number. Your task is to find and return an integer value representing the encoded value of the number.

Input Specification:

input1: An integer value N representing the number to be encoded.

Explanation:

Output Specification:

Return an integer value representing the encoded value of the number.

Example 1:

input1: 34

Output: 916

Here, the given integer is 34, and the square its digit are:

$$3^2 = 9$$

$$4^2 = 16$$

```

1 import java.util.Scanner;
2
3 public class EncodeNumber {
4
5     public static int Encode(int n) {
6         String ans = "";
7
8         while(n != 0){
9             int last = n % 10;
10            ans = (last*last) + ans;
11            n = n / 10;
12        }
13
14        return Integer.parseInt(ans);
15    }
16
17    public static void main(String[] args) {
18        Scanner sc = new Scanner(System.in);
19
20        // Input the number of terms
21        System.out.println("Enter number:");
22        int n = sc.nextInt();
23
24        System.out.println("Encoded number is: " + Encode(n));
25    }
26 }

```

15) Equilibrium Sum

Problem Statement :

Given the function accepts an integer arr of size n as its argument. The function needs to return the index of equilibrium point in the array, where the sum of elements on the left of the index is equal to the sum of elements on the right of the index. If no equilibrium point exists, the function should return -1.

Input arr: {3,4,3,1,6}

Output: 2

Explanation: $3+4 = 7$ and $1+6 = 7$ so 3 is the equilibrium point having index 2.

```
1 import java.util.*;
2 public class EquilibriumSum
3 {
4     public static int equilibrium(int arr[])
5     {
6         int n = arr.length;
7         int leftsum = 0, rightsum = 0;
8         int total = 0;
9
10        for(int i = 0; i < n; i++) {
11            total += arr[i];
12        }
13        for(int i = 0; i < n; i++){
14            rightsum = total - arr[i] - leftsum;
15            if(leftsum == rightsum){
16                return i;
17            }
18            leftsum += arr[i];
19        }
20        return -1;
21    }
22    public static void main(String[] args) {
23        Scanner sc = new Scanner(System.in);
24        System.out.println("Enter no of elements in array: ");
25        int n = sc.nextInt();
26        System.out.println("Enter the elements of array: ");
27        int arr[] = new int[n];
28        for(int i = 0; i < n; i++){
29            arr[i] = sc.nextInt();
30        }
31        System.out.println("Equilibrium Index is: " + equilibrium1(arr));
32    }
33 }
```

16) # Given an array of integers, write a function that finds the missing number.

Input: [1,2,4,5,6]

Output: 3

```

1 def FindMissing(arr):
2     n = len(arr) + 1      # Total number of elements including the missing one
3     TotalSum = n * (n+1) // 2    # Sum of the first n natural numbers
4
5     ActualSum = 0
6     for i in range(len(arr)):
7         ActualSum += arr[i]
8
9     return TotalSum - ActualSum
10
11
12 def main():
13     arr = [1,2,4,5,6]
14     print("Missing number in array is: ", FindMissing(arr))
15
16
17 if __name__ == "__main__":
18     main()

```

17) FirstKwords

Problem Statement :

Print the first K words of the string.

Input: Hello I am a passionate developer
k = 4

Output: Hello I am a

```

1 import java.util.Scanner;
2 public class FirstKWords {
3     public static String firstKWords(String str,int k) {
4         String words[] = str.split("\\s+");
5
6         if(k > words.length){
7             return str;
8         }
9
10        StringBuilder result = new StringBuilder();
11
12        for(int i = 0; i < k; i++){
13            result.append(words[i]);
14            if(i < k-1){
15                result.append(" ");
16            }
17        }
18        return result.toString();
19    }
20
21    public static void main(String[] args) {
22        Scanner sc = new Scanner(System.in);
23
24        System.out.println("Enter String: ");
25        String str = sc.nextLine();
26
27        // Input the number of terms
28        System.out.println("Enter value of K:");
29        int k = sc.nextInt();
30
31        // Print the Fibonacci series
32        System.out.println("The first K words are: "+ firstKWords(str,k));
33    }
34 }

```

18) Floyds Triangle

You have been given an integer N as input . your task is to write a program to print N rows of Floyad's Triangle. Floyd's pattern is a right-angled triangular array of natural numbers , used for the numbering of lines in a printout

.

For N=4,

1

23

456

78910

```
1 import java.util.Scanner;
2
3 public class FloydsTriangle {
4
5     public static void Floyd(int n) {
6         int k = 0;
7         for(int i = 1; i <= n; i++){
8             for(int j = 1; j <= i; j++){
9                 k++;
10                System.out.print(k + " ");
11            }
12            System.out.println();
13        }
14    }
15
16    public static void main(String[] args) {
17        Scanner sc = new Scanner(System.in);
18
19        // Input the number of terms
20        System.out.println("Enter N:");
21        int n = sc.nextInt();
22
23        // Print the Fibonacci series
24        Floyd(n);
25    }
26 }
```

19) Googly Prime no

Problem Statement

A number whose sum of digits is prime.

Input: 43

Output: YES ($4+3 = 7$)

Input: 123

Output: NO ($1+2+3 = 6$)

```
1- import java.util.*;
2
3- public class GooglyPrimeNo{
4
5-     public static boolean isPrime(int n){
6         if(n <= 1)
7             return false;
8
9         for(int i = 2; i <= n/2; i++){
10             if(n % i == 0){
11                 return false;
12             }
13         }
14         return true;
15     }
16
17-     public static String googlyPrime(int n) {
18         int sum = 0;
19         while (n > 0) {
20             int lastDigit = n % 10;
21             sum += lastDigit;
22             n /= 10;
23         }
24         if (isPrime(sum)) {
25             return "YES";
26         }
27         return "NO";
28     }
29
30-     public static void main(String[] args) {
31         Scanner sc = new Scanner(System.in);
32
33         System.out.println("Enter the the decimal number: ");
34         int n = sc.nextInt();
35
36         System.out.println("Output: "+ googlyPrime(n));
37     }
38 }
```

20) Intersection of array

```
1 import java.util.*;
2 public class IntersectionOfArray {
3
4     public static List<Integer> Intersection(int arr1[], int arr2[]){
5         int i = 0, j = 0;
6         List<Integer> ans = new ArrayList<>();
7
8         while(i < arr1.length && j < arr2.length){
9             if(arr1[i] == arr2[j]){
10                 ans.add(arr1[i]);
11                 i++;
12                 j++;
13             }
14             else if(arr1[i] < arr2[j]){
15                 i++;
16             }
17             else{
18                 j++;
19             }
20         }
21         return ans;
22     }
23
24
25     public static void main(String[] args) {
26         Scanner sc = new Scanner(System.in);
27         int[] arr1 = {1, 2, 2, 3, 4};
28         int[] arr2 = {2, 2, 3, 5};
29         List<Integer> result = Intersection(arr1, arr2);
30
31         for (int num : result) {
32             System.out.print(num + " ");
33         }
34     }
35 }
```

21) Largesmall sum

You are required to implement the following Function

def LargeSmallSum(arr)

The function accepts an integers arr of size 'length' as its arguments you are required to return the sum of second largest element from the even positions and second smallest from the odd position of given 'arr'

Assumption:

All array elements are unique

Treat the 0th position as even

NOTE

Return 0 if array is empty

Return 0, if array length is 3 or less than 3

Example

Input

arr:3 2 1 7 5 4

Output

7

Explanation

Second largest among even position elements(1 3 5) is 3

Second smallest among odd position element is 4

Thus output is $3+4 = 7$

Sample Input

arr:1 8 0 2 3 5 6

Sample Output

8

```

1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.Scanner;
4 public class LargeSmallSum {
5
6     public static int largeSmallSum(int[] arr, int n){
7         if (arr == null || n <= 3) {
8             return 0;
9         }
10
11         ArrayList<Integer> even = new ArrayList<Integer>();
12         ArrayList<Integer> odd = new ArrayList<Integer>();
13
14         for (int i = 0; i < n; i++) {
15             if (i % 2 == 0)
16                 even.add(arr[i]);
17             else
18                 odd.add(arr[i]);
19         }
20
21         Collections.sort(even);
22         Collections.sort(odd);
23
24         return even.get(even.size() - 2) + odd.get(1);
25     }
26
27     public static void main(String[] args) {
28         Scanner sc = new Scanner(System.in);
29
30         System.out.println("Enter size of arr: ");
31         int n = sc.nextInt ();
32
33         int arr[] = new int[n];
34         System.out.println("Enter elements: ");
35
36         for (int i = 0; i < n; i++)
37             arr[i] = sc.nextInt ();
38
39         System.out.println ("Sum: " + largeSmallSum(arr, n));
40     }
41 }

```

22) Length Last Word

Problem Statement

Given a string **S** consisting of words and spaces, return the length of the last word in the string.

Input: " I am a passionate Developer "

Output: 9

```

1- import java.util.*;
2
3- public class LengthLastWord{
4-     public static int LengthLast(String s){
5-         String arr[] = s.split(" ");
6
7-         int n = arr.length;
8-         String s1 = arr[n-1];
9
10        return s1.length();
11    }
12
13-    public static void main(String[] args) {
14        Scanner sc = new Scanner(System.in);
15
16        System.out.print("Enter string : ");
17        String s = sc.nextLine();
18
19        System.out.println("Output: " + LengthLast(s));
20    }
21 }

```

23) Linked List Palindrome

Problem Statement

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Input: head = [1, 2, 2, 1]

Output: true

```
import java.util.*;
```

```
public class LinkedListPalindrome {
```

```
    public static Node head;
```

```
    public static Node tail;
```

```
public static class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

```
// Method to reverse the linked list  
public static Node reverse(Node head) {  
    Node curr = head;  
    Node prev = null;  
    Node next = null;  
  
    while (curr != null) {  
        next = curr.next;  
        curr.next = prev;  
        prev = curr;  
        curr = next;  
    }  
    return prev;  
}
```

```
// Method to check if the linked list is a palindrome  
public static boolean isPalindrome(Node head) {  
    if (head == null || head.next == null) {  
        return true; // Single node or empty list is always a palindrome  
    }  
}
```

```
Node slow = head;  
Node fast = head;
```

```
// To find the middle of the linked list  
while (fast != null && fast.next != null) {  
    slow = slow.next;  
    fast = fast.next.next;  
}
```

```
// Reverse the second half of the linked list  
Node revHead = reverse(slow); // Reverse the second half from  
the middle
```

```
Node curr = head;
```

```
// Compare the first half with the reversed second half  
while (revHead != null) {  
    if (curr.data != revHead.data) {  
        return false;  
    }  
    curr = curr.next;  
    revHead = revHead.next;  
}  
return true;  
}
```

```
// Method to add nodes to the linked list  
public static void addNode(int data) {  
    Node newNode = new Node(data);  
    if (head == null) {  
        head = newNode;  
        tail = newNode;  
    } else {
```

```

        tail.next = newNode;
        tail = newNode;
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the number of elements in the linked list:");
    int n = sc.nextInt();

    System.out.println("Enter the elements of the linked list:");
    for (int i = 0; i < n; i++) {
        int element = sc.nextInt();
        addNode(element);
    }

    if (isPalindrome(head)) {
        System.out.println("The linked list is a palindrome.");
    } else {
        System.out.println("The linked list is not a palindrome.");
    }

    sc.close();
}
}

```

24) Longest Substring

Given a string s, find the length of the longest

substring without repeating characters.

Example 1:

Input: s = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3.

```
1- import java.util.*;
2- public class LongestSubstring {
3-
4-     public static int Longest(String s){
5-         Set<Character> set = new HashSet<>();
6-
7-         int maxLength = 0;
8-         int start = 0, end = 0;
9-
10-        while (start < s.length()) {
11-            if (!set.contains(s.charAt(start))) {
12-                set.add(s.charAt(start));
13-                maxLength = Math.max(maxLength, start - end + 1);
14-                start++;
15-            } else {
16-                set.remove(s.charAt(end));
17-                end++;
18-            }
19-        }
20-        return maxLength;
21-    }
22-
23-
24-    public static void main(String[] args) {
25-        Scanner sc = new Scanner(System.in);
26-
27-        System.out.println("Enter string: ");
28-        String str = sc.nextLine();
29-
30-        System.out.println("Output: " + Longest(str));
31-
32-    }
33- }
```

25) Longest Words

Rohan is a kid who has just learned about creating words from alphabets. He has written some words in the notepad of his Father laptop. Now his father wants to find the longest word written by Rohan using a computer program. Write a program to find the longest string in a given list of strings.

Example:

Input: yes no number

Output: The longest string is: number

```

1 import java.util.Scanner;
2
3 public class LongestWord {
4
5     public static String longestWord(String s) {
6         s = s.trim();
7         s = s.replaceAll("\\s+", " ");
8         String arr[] = s.split(" ");
9
10        int max = 0;
11        String longest = "";
12        for(int i = 0; i < arr.length; i++){
13            if(arr[i].length() > max){
14                max = arr[i].length();
15                longest = arr[i];
16            }
17        }
18        return longest;
19    }
20
21    public static void main(String[] args) {
22        Scanner sc = new Scanner(System.in);
23
24        // Input the number of terms
25        System.out.println("Enter string:");
26        String s = sc.nextLine();
27
28        System.out.println("The longest string is: " + longestWord(s));
29    }
30 }

```

26) Find count of magical numbers from 1 to N

A number is magical if:

Convert to binary.

Replace 0 with 1 and 1 with 2 in binary string.

Calculate sum of all digits in binary string.

Resultant must be an odd number

Eg:

Input: N = 5

Output: 2

Explanation:

- 1 -> Binary = 1 -> convert to = 2 sum = 2 (even)
- 2 -> Binary = 10 -> convert to = 21 sum = 3 (odd)
- 3 -> Binary = 11 -> convert to = 22 sum = 4 (even)
- 4 -> Binary = 100 -> convert to = 211 sum = 4 (even)
- 5 -> Binary = 101 -> convert to = 212 sum = 5 (odd)

```
1- import java.util.Scanner;
2
3- public class MagicalNo {
4
5-     public static int Magical(int n) {
6         int count = 0;
7         for (int i = 1; i <= n; i++) {
8             int num = i;
9             int zero = 0;
10
11             while (num != 0) {
12                 if(num % 2 == 0){
13                     zero++;
14                 }
15                 num = num/2;
16             }
17             if(zero % 2 == 1){
18                 count++;
19             }
20         }
21         return count;
22     }
23
24     public static void main(String[] args) {
25         Scanner sc = new Scanner(System.in);
26
27         // Input the number of terms
28         System.out.println("Enter number:");
29         int n = sc.nextInt();
30
31         System.out.println("Output: " + Magical(n));
32     }
33 }
```

27) Matrix

Problem Statement

You are required to input the size of the matrix then the elements of matrix, then you have to divide the main matrix in two sub matrices (even and odd) in such a way that element at 0 index will be considered as even and element at 1st index will be considered as odd and so on. then you have sort the even and odd matrices in ascending order then print the sum of second largest number from both the matrices

Example

enter the size of array : 5
enter element at 0 index : 3
enter element at 1 index : 4
enter element at 2 index : 1
enter element at 3 index : 7
enter element at 4 index : 9
Sorted even array : 1 3 9
Sorted odd array : 4 7

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Scanner;
public class Matrix {

    public static void divideArray(int[] main, ArrayList<Integer> even,
ArrayList<Integer> odd) {
        for (int i = 0; i < main.length; i++) {
            if (i % 2 == 0) {
                even.add(main[i]);
            } else {
```

```
        odd.add(main[i]);
    }
}
}
```

```
public static int secondLargest(ArrayList<Integer> list) {
    if (list.size() < 2) {
        throw new IllegalArgumentException("List must have at least
two elements to find the second largest");
    }
    return list.get(list.size() - 2);
}
```

```
public static int sumOfSecondLargestElements(ArrayList<Integer>
even, ArrayList<Integer> odd) {
```

```
    Collections.sort(even);
    Collections.sort(odd);
```

```
    System.out.println("Sorted even array ");
    for (int e : even) {
        System.out.print(e + " ");
    }
    System.out.println();
```

```
    System.out.println("Sorted odd array ");
    for (int e : odd) {
        System.out.print(e + " ");
    }
    System.out.println();
```

```
    int evenSec = secondLargest(even);
```

```

    int oddSec = secondLargest(odd);

    System.out.println("Second Largest Element in Even List is: " +
evenSec);
    System.out.println("Second Largest Element in Odd List is: " +
oddSec);

    return evenSec + oddSec;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter size of array : ");
    int arrsize = sc.nextInt();

    int[] main = new int[arrsize];
    ArrayList<Integer> even = new ArrayList<>();
    ArrayList<Integer> odd = new ArrayList<>();

    System.out.println("Enter " + arrsize + " Elements");
    for (int i = 0; i < arrsize; i++) {
        main[i] = sc.nextInt();
    }

    divideArray(main, even, odd);

    int sum = sumOfSecondLargestElements(even, odd);
    System.out.println("Sum Of Second Largest Element Of Odd and
Even List: " + sum);
}

```

}

28) Max exponent

Problem Statement

You are given a function,

Int MaxExponents (int a , int b);

You have to find and return the number between 'a' and 'b' (range inclusive on both ends) which has the maximum exponent of 2.

The algorithm to find the number with maximum exponent of 2 between the given range is

Loop between 'a' and 'b'. Let the looping variable be 'i'.
Find the exponent (power) of 2 for each 'i' and store the number with maximum exponent of 2 so far in a variable, let say 'max'.
Set 'max' to 'i' only if 'i' has more exponent of 2 than 'max'.
Return 'max'.

Assumption: $a < b$

Note: If two or more numbers in the range have the same exponents of 2, return the small number.

Example

Input:

7

12

Output:

8

Explanation:

Exponents of 2 in:

7-0

8-3

9-0

10-1

11-0

12-2

Hence maximum exponent if two is of 8.

```

1  #include <iostream>
2  using namespace std;
3  int CountTwos(int n){
4      int count = 0;
5      while(n % 2 == 0){
6          n = n/2;
7          count++;
8      }
9      return count;
10 }
11 int MaxExponents(int a, int b){
12     int maxExponent = -1;
13     int result = a;
14
15     for(int i = a; i <= b; i++){
16         int exponent = CountTwos(i);
17         if(exponent > maxExponent){
18             maxExponent = exponent;
19             result = i;
20         }
21         else if(exponent == maxExponent && i < result){
22             result = i;
23         }
24     }
25     return result;
26 }
27 int main()
28 {
29     int a, b;
30     cout << "Enter the range (a and b): ";
31     cin >> a >> b;
32     int result = MaxExponents(a, b);
33     cout << "Number with maximum exponent of 2 between " << a << " and " << b << " is: " << result << endl;
34     return 0;
35 }

```

29) Max Favourite Song

Problem Statement

Alice has a collection of songs represented as a string S where each character represents a song. A playlist is the substring of the given string with exactly K number of songs. She wants to create a playlist that contains maximum number of her favourite song which is 'a'. Your task is to find and return an integer value

representing the maximum number of favourite songs that she can get in a single playlist.

Input: S = "acdbaaca"

K = 3

Output: 2

Explanation: Substring of S of size 3: {"acd", "cdb", "dba", "baa", "aac", "aca"}

So "a" is coming max 2 times in substring

```

1- import java.util.*;
2
3- public class MaxFavouriteSong{
4-     public static int MaxSong(String s, int k){
5-         int max = 0;
6-         int count = 0;
7-         int i = 0;
8-         int j = -1;
9-         int n = s.length();
10-        char arr[] = s.toCharArray();
11-        for (j = 0; j < k; j++) {
12-            if (arr[j] == 'a') {
13-                count++;
14-            }
15-        }
16-        max = count;
17-        for (j = k; j < n; j++) {
18-            if (arr[i] == 'a') {
19-                count--;
20-            }
21-            i++;
22-
23-            if (arr[j] == 'a') {
24-                count++;
25-            }
26-            max = Math.max(max, count);
27-        }
28-        return count;
29-    }
30-    public static void main(String[] args) {
31-        Scanner sc = new Scanner(System.in);
32
33-        System.out.print("Enter string : ");
34-        String s = sc.nextLine();
35
36-        System.out.println("Enter K: ");
37-        int k = sc.nextInt();
38
39-        System.out.println("Output: "+ MaxSong(s, k));
40-    }
41 }

```

30) Maximum

Problem Statement :

**You are given a function, void MaxInArray(int arr[], int length);
The function accepts an integer array 'arr' of size 'length' as its argument. Implement the function to find the maximum element of the array and print the maximum element and its index to the standard output**

(STDOUT). The maximum element and its index should be printed in separate lines.

Note:

Array index starts with 0

Maximum element and its index should be separated by a line in the output

Assume there is only 1 maximum element in the array

Print exactly what is asked, do not print any additional greeting messages

Example:

Input:

23 45 82 27 66 12 78 13 71 86

Output:

86

9

Explanation:

86 is the maximum element of the array at index 9.

```

1 import java.util.*;
2 public class Maximum
3 {
4     public static void MaxIndElement(int arr[], int length)
5     {
6         int max = arr[0], index = 0;
7         for (int i = 0; i < length; i++){
8             if (arr[i] > max) {
9                 max = arr[i];
10                index = i;
11            }
12        }
13        System.out.println (max);
14        System.out.println (index);
15    }
16
17    public static void main (String[]args)
18    {
19        Scanner sc = new Scanner (System.in);
20
21        System.out.println("Enter size of array: ");
22        int n = sc.nextInt ();
23
24        int arr[] = new int[n];
25
26        System.out.println("Enter elements: ");
27        for (int i = 0; i < n; i++)
28            arr[i] = sc.nextInt ();
29
30        System.out.println("Maximum element with its index is: ");
31        MaxIndElement(arr, arr.length);
32    }
33 }

```

32) Maximum With Index

Given an array of integers, write a function that finds the maximum element and its index.

Input: [1,8,4,9,6]

Output: (9,3)

```

1 def MaximumIndex(arr):
2     max = arr[0]
3     maxIndex = 0
4
5     for i in range(1, len(arr)):
6         if arr[i] > max:
7             max = arr[i]
8             maxIndex = i
9
10    return (max, maxIndex)
11
12
13 def main():
14     arr = [1,8,4,9,6]
15     print("Maximum number with its index is: ", MaximumIndex(arr))
16
17
18 if __name__ == "__main__":
19     main()

```

33) Merge Sorted Arrays

Problem Statement :

Given two arrays of integers, return merged sorted array.

Input: arr1 = [1, 2, 3, 4, 5],
arr2 = [2, 4, 6, 8, 10]

Output: [1, 2, 2, 3, 4, 4, 5, 6, 8, 10]

```

1 import java.util.*;
2 public class MergeSortedArrays
3 {
4     public static List<Integer> MergeSortedArray(int arr1[], int arr2[])
5     {
6         List<Integer> merged = new ArrayList<>();
7         int i = 0, j = 0;
8         while (i < arr1.length && j < arr2.length) {
9             if (arr1[i] < arr2[j]) {
10                 merged.add(arr1[i]);
11                 i++;
12             } else {
13                 merged.add(arr2[j]);
14                 j++;
15             }
16         }
17         while(i < arr1.length){
18             merged.add(arr1[i]);
19             i++;
20         }
21         while(j < arr2.length){
22             merged.add(arr2[j]);
23             j++;
24         }
25         return merged;
26     }
27     public static void main(String[] args) {
28         int[] arr1 = {1, 3, 5, 7};
29         int[] arr2 = {2, 4, 6};
30
31         List<Integer> result = MergeSortedArray(arr1, arr2);
32         for (int num : result) {
33             System.out.print(num + " ");
34         }
35         System.out.println(); // Output: 1 2 3 4 5 6 7
36     }
37 }

```

34) Most Frequent Vowel

You are given a string `str` of length `n`. You have to find the most frequent vowel in the string `str`

Note: You may assume that `str` will always have a unique most frequent vowel.

Sample Test Case:

Input:

6 -> string length
xyuaab

Output:

a

Explanation: As the vowel 'a' occurs the most in the string str, hence 'a' is printed in the output.

```
1 def MostFrequentVowel(str):
2     FreqVow = {}
3     mostFreqVow = None
4     maxVow = 0
5
6     for ch in str:
7         if ch in 'aeiou':
8             FreqVow[ch] = FreqVow.get(ch,0)+1
9
10            if FreqVow[ch] > maxVow:
11                maxVow = FreqVow[ch]
12                mostFreqVow = ch
13
14     return mostFreqVow
15
16
17 def main():
18     str = "xyuaab"
19     print("Most Frequent vowel is: ", MostFrequentVowel(str))
20
21
22 if __name__ == "__main__":
23     main()
```

35) Most Hyphen

Implement the following functions.a

char*MoveHyphen(char str[],int n);

The function accepts a string “str” of length ‘n’, that contains alphabets and hyphens (-). Implement the function to move all hyphens(-) in the string to the front of the given string.

NOTE:- Return null if str is null.

Example :-

Input:

str.Move-Hyphens-to-Front

Output:

—MoveHyphenstoFront

Explanation:-

The string “Move-Hyphens -to-front” has 3 hyphens (-), which are moved to the front of the string, this output is “— MoveHyphen”

Sample Input

Str: String-Compare

Sample Output-

-StringCompare

```

1 import java.util.*;
2 public class MoveHyphen {
3
4     public static String MHyphen(String str, int n){
5         if (str == null) {
6             return null;
7         }
8
9         StringBuilder hyphens = new StringBuilder();
10        StringBuilder nonHyphens = new StringBuilder();
11
12        for (int i = 0; i < n; i++) {
13            if (str.charAt(i) == '-') {
14                hyphens.append('-');
15            } else {
16                nonHyphens.append(str.charAt(i));
17            }
18        }
19
20        return hyphens.toString() + nonHyphens.toString();
21    }
22
23    public static void main(String[] args) {
24        Scanner sc = new Scanner(System.in);
25
26        System.out.println("Enter string with hyphen: ");
27        String str = sc.next ();
28
29        System.out.println ("Result: " + MHyphen(str, str.length()));
30    }
31 }

```

36) Negative Stock Price

Problem Statement

You are working on a financial analysing tool which represents daily stock price of a company over time. Each element in an integer array **A** of size **N** represents the closing price of the stock for that particular day. Your task is to find and return an integer value representing the total number of days where the stock market price decreased indicating negative growth.

Input: N = 6, A[] = {2,3,1,4,5,2}

Output: 2

Input: N = 1, A[] = {6}

Output: 0

```
1 import java.util.*;
2
3 public class NegativeStockPrice{
4     public static int CountNegativePrice(int arr[], int N){
5         int count = 0;
6
7         if(N <= 1){
8             return 0;
9         }
10
11         for(int i = 0; i < N-1; i++){
12             if(arr[i+1] < arr[i]){
13                 count++;
14             }
15         }
16         return count;
17     }
18
19     public static void main(String[] args) {
20         Scanner sc = new Scanner(System.in);
21
22         System.out.print("Enter size of array : ");
23         int n = sc.nextInt();
24
25         System.out.println("Enter elements: ");
26         int arr[] = new int[n];
27         for (int i = 0; i < n; i++) {
28             arr[i] = sc.nextInt();
29         }
30
31         System.out.println("Output: "+ CountNegativePrice(arr, n));
32     }
33 }
```

37) Operation Choices

Problem Statement

You are required to implement the following function.

Int OperationChoices(int c, int n, int a , int b)

The function accepts 3 positive integers 'a' , 'b' and 'c ' as its arguments. Implement the function to return.

(a+ b) , if c=1

(a – b) , if c=2

(a * b) , if c=3

(a / b) , if c =4

Assumption : All operations will result in integer output.

Example:

Input

c :1

a:12

b:16

Output:

Since 'c'=1 , (12+16) is performed which is equal to 28 , hence 28 is returned.

Sample Input:

c : 2

a : 16

b : 20

Sample Output:

-4

```

1  #include <iostream>
2  using namespace std;
3  int operationChoices(int c, int a, int b)
4  {
5      if (c == 1) {
6          return a + b;
7      }
8      else if (c == 2) {
9          return a - b;
10     }
11     else if (c == 3) {
12         return a * b;
13     }
14     else if (c == 4) {
15         return a / b;
16     }
17     return 0;
18 }
19 int main()
20 {
21     int x, y, z;
22     int result;
23     cout<<"Enter value of c: "<<endl;
24     cin >> x;
25
26     cout<<"Enter value of a: "<<endl;
27     cin >> y;
28
29     cout<<"Enter value of b: "<<endl;
30     cin >> z;
31
32     result = operationChoices(x, y, z);
33     cout <<"Result: "<<result;
34     return 0;
35 }
36

```

38) Pair Sum Max Product

Given Array of size N, We have to return the pair whose sum is equal to target and having maximum product.

Note: First value of pair must be greater than the second value

Input: Target: 18

N = 8

arr = [11,1,2,8,10,11,15,7]

Output: [10, 8] Sum is 18 and product is 80 which is maximum

```
import java.util.*;
```

```
public class PairSumMaxProduct{
```

```
    public static void sortDesc(int arr[]){
```

```
        Arrays.sort(arr);
```

```
        int n = arr.length;
```

```
        for(int i = 0; i < n/2; i++){
```

```
            int temp = arr[i];
```

```
            arr[i] = arr[n-i-1];
```

```
            arr[n-i-1] = temp;
```

```
        }
```

```
    }
```

```
    public static int[] PairSum(int n, int arr[], int target){
```

```
        int ans[] = new int[2];
```

```
        int prod = 0;
```

```
        sortDesc(arr);
```

```

int start = 0;
int end = n-1;

while(start < end){

    int sum = arr[start] + arr[end];

    if(sum == target){
        int currProd = arr[start] * arr[end];
        if(currProd > prod){
            prod = currProd;
            ans[0] = arr[start];
            ans[1] = arr[end];
        }
        start++;
        end--;
    }
    else if(sum < target){
        end--;
    }
    else{
        start++;
    }
}
return ans;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

```

```

System.out.print("Enter size of array : ");
int n = sc.nextInt();

System.out.println("Enter elements: ");
int arr[] = new int[n];
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

System.out.println("Enter target: ");
int target = sc.nextInt();

int[] result = PairSum(n, arr, target);
if (result[0] == 0 && result[1] == 0) {
    System.out.println("No valid pair found.");
} else {
    System.out.println("Output: [" + result[0] + ", " + result[1] + "]");
}
}
}

```

39) Password Checker

You are given a function.

```
int CheckPassword(char str[], int n);
```

The function accepts string str of size n as an argument.

Implement the function which returns 1 if given string str is valid password else 0.

str is a valid password if it satisfies the below conditions.

- At least 4 characters

- At least one numeric digit**
- At Least one Capital Letter**
- Must not have space or slash (/)**
- Starting character must not be a number**

Assumption:

Input string will not be empty.

Example:

Input 1:

aA1_67

Input 2:

a987 abC012

Output 1:

1

Output 2:

0

```

1- import java.util.Scanner;
2- public class PasswordChecker {
3
4-     public static int checkPassword(String str, int n){
5-         if(n < 4){
6-             return 0;
7-         }
8-         if(str.charAt(0) >= '0' && str.charAt(0) <= '9'){
9-             return 0;
10-        }
11-        int count = 0;
12-        for(int i = 0; i < n; i++){
13-            if(str.charAt(i) == ' ' || str.charAt(i) == '/'){
14-                return 0;
15-            }
16-            if(str.charAt(i) >= '0' && str.charAt(i) <= '9'){
17-                count++;
18-            }
19-            if(str.charAt(i) >= 'A' && str.charAt(i) <= 'Z'){
20-                count++;
21-            }
22-        }
23-        if(count < 2){
24-            return 0;
25-        }
26-
27-        return 1;
28-    }
29
30-    public static void main(String[] args) {
31-        Scanner sc = new Scanner(System.in);
32
33-        System.out.println("Enter Password: ");
34-        String str = sc.next ();
35
36-        System.out.println (checkPassword (str, str.length ()));
37-    }
38- }

```

40) Print Even Odd

Problem Statement

Jack has an array of length N. He wants to label whether the number in the array is even or odd. Your task is to help find and return a string with labels even or odd in sequence according to which the numbers appear in the array.

Input: N = 6

arr = [1, 2, 3, 4, 5, 6]

Output: "odd even odd even odd even"

```
1- import java.util.*;
2
3- public class PrintEvenOdd{
4-     public static String EvenOdd(int arr[], int n){
5-         if(n == 0){
6-             return "";
7-         }
8-         String result = "";
9-         for(int i = 0; i < n; i++){
10-             if(arr[i] % 2 == 0){
11-                 result += "even ";
12-             }
13-             else{
14-                 result += "odd ";
15-             }
16-         }
17-         return result;
18-     }
19
20-     public static void main(String[] args) {
21-         Scanner sc = new Scanner(System.in);
22
23-         System.out.print("Enter size of array : ");
24-         int n = sc.nextInt();
25
26-         System.out.println("Enter elements: ");
27-         int arr[] = new int[n];
28-         for (int i = 0; i < n; i++) {
29-             arr[i] = sc.nextInt();
30-         }
31
32-         System.out.println("Output: "+ EvenOdd(arr, n));
33-     }
34 }
```

41) Product Smallest

Implement the following Function

```
def ProductSmallestPair(sum, arr)
```

The function accepts an integers sum and an integer array arr of size n. Implement the function to find the pair, (arr[j], arr[k]) where $j \neq k$, Such that arr[j] and arr[k] are the least two elements of array (arr[j] + arr[k] $\leq$ sum) and return the product of element of this pair

NOTE

Return -1 if array is empty or if $n < 2$

Return 0, if no such pairs found

All computed values lie within integer range

Example

Input

sum:9

size of Arr = 7

Arr:5 2 4 3 9 7 1

Output

2

Explanation

Pair of least two element is (2, 1) $2 + 1 = 3 < 9$, Product of (2, 1) $2 * 1 = 2$. Thus, output is 2

Sample Input

sum:4

size of Arr = 6

Arr:9 8 3 -7 3 9

Sample Output

-21

```

1- import java.util.*;
2- public class ProductSmallest {
3
4-     public static int ProductSmallestPair(int[]arr, int n, int sum){
5-         if (arr == null || n < 2) {
6-             return 0;
7-         }
8
9-         Arrays.sort(arr);
10
11         int checkSum = arr[0] + arr[1];
12         if(checkSum <= sum){
13             return arr[0] * arr[1];
14         }else{
15             return 0;
16         }
17     }
18
19     public static void main(String[] args) {
20         Scanner sc = new Scanner(System.in);
21
22         System.out.println("Enter sum: ");
23         int sum = sc.nextInt ();
24
25         System.out.println("Enter size of arr: ");
26         int n = sc.nextInt ();
27
28         int arr[] = new int[n];
29         System.out.println("Enter elements: ");
30
31         for (int i = 0; i < n; i++)
32             arr[i] = sc.nextInt ();
33
34         System.out.println ("Sum: " + ProductSmallestPair(arr, n, sum));
35     }
36 }

```

42) Rat Count House

Problem Description :

The function accepts two positive integers 'r' and 'unit' and a positive integer array 'arr' of size 'n' as its argument 'r' represents the number of rats present in an area, 'unit' is the amount of food each rat consumes and each ith element of array

'arr' represents the amount of food present in 'i+1' house number, where $0 \leq i$

Note:

- Return -1 if the array is null**
- Return 0 if the total amount of food from all houses is not sufficient for all the rats.**
- Computed values lie within the integer range.**

Example:

Input:

r: 7

unit: 2

n: 8

arr: 2 8 3 5 7 4 1 2

Output:

4

Explanation:

**Total amount of food required for all rats = $r * \text{unit}$
 $= 7 * 2 = 14$.**

The amount of food in 1st houses = $2+8+3+5 = 18$. Since, amount of food in 1st 4 houses is sufficient for all the rats. Thus, output is 4.

```

1- import java.util.*;
2
3- public class RatCountHouse {
4
5-     public static int Solve(int r, int unit, int arr[], int n){
6-         if(arr == null){
7-             return 0;
8-         }
9
10-         int totalFood = r * unit;
11
12-         int sum = 0, count = 0;
13
14-         for(int i = 0; i < n; i++){
15-             sum += arr[i];
16-             count++;
17-             if(sum >= totalFood){
18-                 break;
19-             }
20-         }
21-         if(sum < totalFood){
22-             return 0;
23-         }
24
25-         return count;
26-     }
27
28-     public static void main(String[] args) {
29-         Scanner sc = new Scanner(System.in);
30
31-         int r = sc.nextInt ();
32-         int unit = sc.nextInt ();
33-         int n = sc.nextInt ();
34-         int arr[] = new int[n];
35
36-         for (int i = 0; i < n; i++){
37-             arr[i] = sc.nextInt ();
38-         }
39
40-         System.out.println(Solve(r, unit, arr, n));
41-     }
42- }

```

43)Rearrangement of bits

Rearrangement Of Bits

Alex Gives You a positive Number N and wants you to rearrange the

bits of the number in its binary representation such that all set bits are in consecutive order. Your task is to find and return an integer value representing the minimum possible number that can be formed after re-arranging the bits of the number N.

Example

Input1: 10

Output: 3

Explanation: 10 -> binary: 1010 count the set bits and arrange in consecutive order such as 0011 which in decimal is 3.

Input: 2

Output: 1

```

1  #include<iostream>
2  using namespace std;
3
4  int rearrangement(int n){
5      int count = 0;
6      while(n > 0){
7          if(n & 1) {
8              count++;
9          }
10         n = n >> 1;
11     }
12
13     return (1 << count)-1;
14 }
15
16 int main(){
17     int n;
18
19     cout<<"Enter the number: "<<endl;
20     cin>>n;
21
22     cout<<"Output: "<<rearrangement(n)<<endl;
23
24 }

```

44) Repeat a string

Given an integer N and string S. your task is to find and return new String which consist of the original string repeated N times.

Input: N = 3, S = "abc"

Output: "abcbcabcb"

```

1 import java.util.*;
2 public class RepeatString {
3
4     public static String repeate(String str, int n){
5         String repeate = "";
6         while(n > 0){
7             repeate += str;
8             n--;
9         }
10        return repeate;
11    }
12
13
14    public static void main(String[] args) {
15        String str = "abc";
16        int n = 3;
17        System.out.println(repeate(str, n));
18    }
19 }

```

45) Replace Character

You are given a function,

Void *ReplaceCharacter(Char str[], int n, char ch1, char ch2);

The function accepts a string 'str' of length n and two characters 'ch1' and 'ch2' as its arguments . Implement the function to modify and return the string 'str' in such a way that all occurrences of 'ch1' in original string are replaced by 'ch2' and all occurrences of 'ch2' in original string are replaced by 'ch1'.

Assumption: String Contains only lower-case alphabetical letters.

Note:

Return null if string is null.

If both characters are not present in string or both of them are same , then return the string unchanged.

Example:

Input:

Str: apples

ch1:a

ch2:p

Output:

paales

Explanation:

‘A’ in original string is replaced with ‘p’ and ‘p’ in original string is replaced with ‘a’, thus output is paales.

```

1  #include<iostream>
2  #include<cstring>
3  using namespace std;
4
5  void ReplaceCharacter(char str[], int n, char ch1, char ch2){
6      for(int i = 0; i < n; i++){
7          if(str[i] == ch1){
8              str[i] = ch2;
9          }
10         else if(str[i] == ch2){
11             str[i] = ch1;
12         }
13     }
14     cout<<str;
15
16
17  int main(){
18
19     char str[100];
20     char b, c;
21     int len;
22
23     cout<<"Enter string: "<<endl;
24     cin >> str;
25
26     cout<<"Enter ch1: "<<endl;
27     cin >> b;
28
29     cout<<"Enter ch2: "<<endl;
30     cin >> c;
31
32     len = strlen(str);
33     ReplaceCharacter(str, len, b, c);
34     return 0;
35 }

```

46) Replace of frequent characters

```

1 import java.util.*;
2 public class ReplaceFrequentCharacters {
3
4     static String Replace(String str, char c){
5         Map<Character, Integer> mp = new HashMap<>();
6
7         for(int i = 0; i < str.length(); i++){
8             if(mp.containsKey(str.charAt(i))){
9                 mp.put(str.charAt(i), mp.get(str.charAt(i))+1);
10            }
11            else{
12                mp.put(str.charAt(i),1);
13            }
14        }
15        int max = 0;
16        char maxChar = ' ';
17
18        for(Map.Entry<Character, Integer> entry : mp.entrySet()){
19            if(entry.getValue() > max){
20                max = entry.getValue();
21                maxChar = entry.getKey();
22            }
23        }
24
25        return str.replace(maxChar, c);
26    }
27
28    public static void main(String[] args) {
29        Scanner sc = new Scanner(System.in);
30
31        String str = "bbadbbbababb";
32        String result = Replace(str, 't');
33        System.out.println(result);
34    }
35 }
36 }

```

47) Reverse String

```

1 import java.util.Scanner;
2 public class ReverseString {
3
4     public static String reverseString(String str){
5         String reversed = "";
6         for(int i = 0; i < str.length(); i++){
7             reversed = str.charAt(i) + reversed;
8         }
9         return reversed;
10    }
11
12
13    public static void main(String[] args) {
14        String str = "hello";
15        System.out.println(reverseString(str));
16    }
17 }

```

48) Reverse Words

Input:

A single line of text containing words separated by spaces. The input string consists of only printable ASCII characters.

Output:

The string with words reversed in order.

Example:

Input: Hello

World

Output:

World Hello

```

1 import java.util.*;
2
3 public class ReverseWords{
4
5     public static void reverseWord(String s){
6
7         String[] words = s.split(" ");
8
9         String reversedString = "";
10
11         for (int i = words.length - 1; i >= 0; i--) {
12             reversedString += words[i];
13             if (i != 0) {
14                 reversedString += " "; // Add space between words
15             }
16         }
17
18         System.out.println(reversedString);
19     }
20
21     public static void main(String[] args) {
22         Scanner sc = new Scanner(System.in);
23
24         System.out.println("Enter string: ");
25         String s=sc.nextLine();
26
27         reverseWord(s);
28     }
29 }

```

49) Roots

Function to find the roots of a quadratic equation $ax^2 + bx + c = 0$

Formula: $+X = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$

$-X = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$

```

1 import math
2
3 def RootsofEquation(a, b, c):
4     root1 = (-b + math.sqrt(b * b - 4 * a * c)) / (2 * a)
5     root2 = (-b - math.sqrt(b * b - 4 * a * c)) / (2 * a)
6     return root1, root2
7
8
9 # Example usage
10 a = 1
11 b = -3
12 c = 2
13 result = RootsofEquation(a, b, c)
14 print("Roots of the equation:", result)

```

50) Rotate array by K

Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

Example 1:

Input: nums = [1,2,3,4,5,6,7], k = 3

Output: [5,6,7,1,2,3,4]

Explanation:

rotate 1 steps to the right: [7,1,2,3,4,5,6]

rotate 2 steps to the right: [6,7,1,2,3,4,5]

rotate 3 steps to the right: [5,6,7,1,2,3,4]

```
import java.util.*;
```

```
public class RotateArrayByK{
    public static void rotate(int arr[], int n, int k){
        k = k % n;
        swap(arr, 0, n-1);
        swap(arr, 0, k-1);
        swap(arr, k, n-1);
    }
}
```

```
public static void swap(int arr[], int s, int e){
    while(s <= e){
        int temp = arr[s];
        arr[s] = arr[e];
        arr[e] = temp;
        s++;
        e--;
    }
}
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter size of arr: ");
    int n = sc.nextInt();

    System.out.print("Enter elements : ");
    int arr[] = new int[n];

    for(int i = 0; i < n; i++){
        arr[i] = sc.nextInt();
    }

    System.out.println("Enter K: ");
}
```

```
int k = sc.nextInt();

rotate(arr, n, k);

System.out.println("After rotating array by k steps: ");
for(int i = 0; i < n; i++){
    System.out.print(arr[i] + " ");
}

}

}
```

51) Second largest

```

1 import java.util.Scanner;
2 public class SecondLargest {
3
4     public static int secondLargest(int arr[]){
5         int first = Integer.MIN_VALUE;
6         int second = Integer.MIN_VALUE;
7
8         for (int i = 0; i < arr.length; i++) {
9             if(arr[i] > first){
10                 second = first;
11                 first = arr[i];
12             }
13             else if(arr[i] > second && arr[i] != first){
14                 second = arr[i];
15             }
16         }
17         return second;
18     }
19
20     public static void main(String[] args) {
21         int[] arr = {1, 3, 5, 2, 4, 6, 8};
22         System.out.println(secondLargest(arr));
23     }
24 }

```

52) Set Zero Matrix

Problem Statement

Given an $m * n$ integer matrix , if an element is 0, set its entire row and column to 0.

Input: matrix = [[1,1,1],
[1,0,1],
[1,1,1]]

Output: matrix = [[1,0,1],

[0,0,0],
[1,0,1]]

```
import java.util.*;
```

```
public class SetZeroMatrix{  
    public static void setZero(int matrix[][]){  
        int rowlen = matrix.length;  
        int collen = matrix[0].length;  
  
        int rowarr[] = new int[rowlen];  
        int colarr[] = new int[collen];  
  
        for(int i = 0; i < rowlen; i++){  
            for(int j = 0; j < collen; j++){  
                if(matrix[i][j] == 0){  
                    rowarr[i] = 1;  
                    colarr[j] = 1;  
                }  
            }  
        }  
  
        for(int i = 0; i < rowlen; i++){  
            for(int j = 0; j < collen; j++){  
                if(rowarr[i] == 1 || colarr[j] == 1){  
                    matrix[i][j] = 0;  
                }  
            }  
        }  
    }  
}
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter no of row : ");
    int row = sc.nextInt();

    System.out.print("Enter no of col : ");
    int col = sc.nextInt();

    System.out.println("Enter elements: ");
    int matrix[][] = new int[row][col];
    for (int i = 0; i < row; i++) {
        for (int j = 0; j < col; j++) {
            matrix[i][j] = sc.nextInt();
        }
    }

    setZero(matrix);

    System.out.println("Output matrix: ");
    for (int i = 0; i < row; i++) {
        for (int j = 0; j < col; j++) {
            System.out.print(matrix[i][j] + " ");
        }
        System.out.println();
    }
}
}

```

53) Small large sum

Write a function `SmallLargeSum(array)` which accepts the array as an argument or parameter, that performs the addition of the second largest element from the even location with the second largest element from an odd location?

Rules:

- a. All the array elements are unique.**
- b. If the length of the array is 3 or less than 3, then return 0.**
- c. If Array is empty then return zero.**

Sample Test Case 1:

Input:

6

3 2 1 7 5 4

Output:

7

Explanation: The second largest element in the even locations (3, 1, 5) is 3. The second largest element in the odd locations (2, 7, 4) is 4. So the addition of 3 and 4 is 7. So the answer is 7.

Sample Test Case 2:

Input:

7

4 0 7 9 6 4 2

Output:

10

```

1 public class SmallLargeSum {
2     public static int SmallLargeSum(int[] array) {
3         if (array.length <= 3) {
4             return 0;
5         }
6         int firstEven = Integer.MIN_VALUE;
7         int secondEven = Integer.MIN_VALUE;
8         int firstOdd = Integer.MIN_VALUE;
9         int secondOdd = Integer.MIN_VALUE;
10        for (int i = 0; i < array.length; i++) {
11            if (i % 2 == 0) {
12                if (array[i] > firstEven) {
13                    secondEven = firstEven;
14                    firstEven = array[i];
15                } else if (array[i] > secondEven) {
16                    secondEven = array[i];
17                }
18            } else {
19                if (array[i] > firstOdd) {
20                    secondOdd = firstOdd;
21                    firstOdd = array[i];
22                } else if (array[i] > secondOdd) {
23                    secondOdd = array[i];
24                }
25            }
26        }
27        return secondEven + secondOdd;
28    }
29
30    public static void main(String[] args) {
31        int[] array1 = {3, 2, 1, 7, 5, 4};
32        System.out.println(SmallLargeSum(array1)); // Output: 7
33
34        int[] array2 = {4, 0, 7, 9, 6, 4, 2};
35        System.out.println(SmallLargeSum(array2)); // Output: 10
36    }
37 }

```

54) Standard deviation

```

1 import java.util.*;
2 public class StandardDeviation {
3
4     public static double SD(List<Integer> arr){
5
6         int n = arr.size();
7         double avg = 0;
8         double sum = 0;
9
10
11         for(int i = 0; i < n; i++){
12             sum += arr.get(i);
13         }
14         avg = sum / n;
15
16         double sumOfSquare = 0;
17         for(int i = 0; i < n; i++){
18             sumOfSquare += Math.pow(arr.get(i) - avg, 2);
19         }
20         double sd = Math.sqrt(sumOfSquare / n);
21         return sd;
22     }
23     public static void main(String[] args) {
24         Scanner sc = new Scanner(System.in);
25
26         System.out.println("Enter the no of elements: ");
27         int n = sc.nextInt();
28         List<Integer> arr = new ArrayList<Integer>();
29         System.out.println("Enter elements: ");
30         for(int i = 0; i < n; i++){
31             arr.add(sc.nextInt());
32         }
33         System.out.println(SD(arr));
34     }
35 }

```

55) String Decoder

Problem Statement

You are provided with a string which has a sequence of 1s and Os. This sequence is the encoded version of a english word. You are supposed to write a program to decode the provided string

and find the original word. Each uppercase Alphabet is representing by a sequence of 1s

Input: 10110111

Output: ABC

Explanation: 1 0 1 1 0 1 1 1

A B C

```

1 import java.util.*;
2 public class StringDecoder {
3
4     static String Decoder(String s){
5         if (s.length() == 0 || s.charAt(0) == '0') {
6             return "";
7         }
8
9         String ans = "";
10        int count = 0;
11
12        for(int i = 0; i < s.length(); i++){
13            if(s.charAt(i) == '1'){
14                count++;
15            }
16            else{
17                ans += (char)('A' + count - 1);
18                count = 0;
19            }
20        }
21        ans += (char)('A' + count - 1);
22        return ans;
23    }
24
25
26    public static void main(String[] args) {
27        Scanner sc = new Scanner(System.in);
28
29        System.out.println("Enter string: ");
30        String str = sc.nextLine();
31
32        System.out.println("Output: " + Decoder(str));
33
34    }
35 }

```

56) Sum 3 and 5

Problem Statement

You are required to implement the following function:

Int Calculate(int m, int n);

The function accepts 2 positive integer 'm' and 'n' as its arguments. You are required to calculate the sum of numbers divisible both by 3 and 5, between 'm' and 'n' both inclusive and return the same.

Note

$0 < m \leq n$

Example

Input:

m : 12

n : 50

Output

90

Explanation:

The numbers divisible by both 3 and 5, between 12 and 50 both inclusive are {15, 30, 45} and their sum is 90.

Sample Input

m : 100

n : 160

Sample Output

510

```

1  #include <stdio.h>
2
3  int Calculate(int m, int n){
4      int sum = 0;
5      for(int i = m; i <= n; i++){
6          if(i % 3 == 0 && i % 5 == 0){
7              sum += i;
8          }
9      }
10     return sum;
11 }
12
13
14 int main()
15 {
16     int m, n, result;
17
18     printf ("Enter the value of m : ");
19     scanf ("%d", &m);
20
21     printf ("Enter the value of n : ");
22     scanf ("%d", &n);
23
24     result = Calculate (m, n);
25     printf ("%d", result);
26
27     return 0;
28 }

```

57) Sum Binary

You are given an integer 'n'. Write a Python function to calculate and return the sum of the digits in 'n' after converting it to its binary representation.

For example, 15, which has a binary representation of 1111, should

return 4.

```

1  def SumBinary(n):
2      sum = 0
3
4      while n > 0:
5          lastDigit = n & 1
6          sum += lastDigit
7          n = n >> 1
8
9      return sum
10
11
12 def main():
13     print("Enter number: ")
14     n = int(input())
15     result = SumBinary(n)
16     print("Sum of binary digits:", result)
17
18 if __name__ == "__main__":
19     main()

```

58) Sum Even Index

Problem Statement

Given an array A of length N, find the sum of even positions after reversing the array. Your task is to find and return an integer value representing sum of the array elements present at the even index of the reversed array.

input: N = 6, arr = 10,20,30,40,50,60

output: 120

Explanation: reversed array: 60,50,40,30,20,10 and then sum even index(index starting from 0) elements which are 60,40,20

```

1 import java.util.*;
2 public class SumEvenIndex{
3     public static int sumEvenIndex(int arr[], int n){
4         if(n == 0){
5             return 0;
6         }
7         int i = 0, j = n-1;
8         while(i <= j){
9             int temp = arr[i];
10            arr[i] = arr[j];
11            arr[j] = temp;
12            i++;
13            j--;
14        }
15        int sum = 0;
16        for(int k = 0; k < n; k++){
17            if(k % 2 == 0){
18                sum += arr[k];
19            }
20        }
21        return sum;
22    }
23    public static void main(String[] args) {
24        Scanner sc = new Scanner(System.in);
25
26        System.out.print("Enter size of array : ");
27        int n = sc.nextInt();
28
29        System.out.println("Enter elements: ");
30        int arr[] = new int[n];
31        for (int i = 0; i < n; i++) {
32            arr[i] = sc.nextInt();
33        }
34        System.out.println("Output: "+ sumEvenIndex(arr, n));
35    }
36 }

```

59) Sum of divisors

Input: 12

// Output: 28

// Explanation: Divisors of 12 between 1 to 12 are: 1,2,3,4,6,12 so by adding them we get 28.

```
1  #include <iostream>
2  using namespace std;
3
4  int SumOfDivisor(int N) {
5      int sum = 0;
6      for (int i = 1; i <= N; i++) {
7          if (N % i == 0) {
8              sum += i;
9          }
10     }
11     return sum;
12 }
13
14 int main() {
15     int N = 12;
16     cout << "The sum of the divisors of " << N << " is " << SumOfDivisor(N) << endl;
17     return 0;
18 }
```

60) Sum Prime No

Problem Statement :

Write a function that takes an integer n as input and returns the sum of all prime numbers less than N.

Input: 10

Output: 17

Explanation: prime no less than 10 are: 2, 3, 5, 7

```

1 import java.util.*;
2 public class SumPrimeNo
3 {
4     public static int sumPrime(int n) {
5         int sum = 0;
6
7         for (int i = 2; i < n; i++) {
8             boolean isPrime = true;
9
10            if (i <= 1) {
11                isPrime = false;
12            }
13            else {
14                for (int j = 2; j <= Math.sqrt(i); j++) {
15                    if (i % j == 0) {
16                        isPrime = false;
17                        break;
18                    }
19                }
20            }
21            if (isPrime) {
22                sum += i;
23            }
24        }
25        return sum;
26    }
27    public static void main(String[] args) {
28        Scanner sc = new Scanner(System.in);
29
30        System.out.println("Enter N: ");
31        int n = sc.nextInt();
32
33        System.out.println("Output: " + sumPrime(n));
34    }
35 }

```

61) Table

Problem: Write a program in C to display the table of a number and print the sum of all the multiples in it.

Test Cases:

Test Case 1:

Input:

5

Expected Result Value:

5, 10, 15, 20, 25, 30, 35, 40, 45, 50
275

Test Case 2:

Input:

12

Expected Result Value:

12, 24, 36, 48, 60, 72, 84, 96, 108, 120
660

```
1  #include<stdio.h>
2  int main ()
3  {
4      int n, i, value = 0, sum = 0;
5
6      printf ("Enter the number for which you want to know the table : ");
7      scanf ("%d", &n);
8
9      for (i = 1; i <= 10; ++i){
10         value = n * i;
11         printf ("%d ", value);
12         sum = sum + value;
13     }
14
15     printf ("\nSum is %d", sum);
16
17     return 0;
18 }
```

62) Vowel Permutation

Problem Statement

You are given a string S and your task is to find and return the count of permutation formed by fixing the positions of the vowels present in the string.

Input: ABC

Output: 2

Explanation: A will be constant, and remaining 2 will return their permutation which is factorial of 2

```

1 import java.util.*;
2 public class VowelPermutation{
3
4     public static int factorial(int n){
5         if(n == 0 || n == 1){
6             return 1;
7         }
8         int ans = 1;
9         for(int i = 2; i <= n; i++){
10             ans *= i;
11         }
12         return ans;
13     }
14     public static int permutation(String s){
15         HashSet<Character> vowels = new HashSet<>();
16
17         vowels.add('A');
18         vowels.add('E');
19         vowels.add('I');
20         vowels.add('O');
21         vowels.add('U');
22         vowels.add('a');
23         vowels.add('e');
24         vowels.add('i');
25         vowels.add('o');
26         vowels.add('u');
27
28         int nonVowelCount = 0;
29         for(int i = 0; i < s.length(); i++){
30             if(!vowels.contains(s.charAt(i))){
31                 nonVowelCount++;
32             }
33         }
34         return factorial(nonVowelCount);
35     }

```

63) Vowel repetition

You are a given a string str of length n. You have to find the most frequent vowel in the string str

Note: You may assume that str will always hav a unique most frequent vowel.

Sample Test Case:

Input:

7 -> string length

xayuaba

Output:

a

Explanation: As the vowel 'a' occurs the most in the string str, hence 'a' is printed in the output.

```

1 import java.util.HashMap;
2
3 public class VowelRepetition {
4
5     // Method to find the most frequent vowel in the string
6     public static char MostFrequentVowel(String str) {
7         HashMap<Character, Integer> vowelCounts = new HashMap<>();
8         char mostFrequentVowel = '\0';
9         int maxCount = 0;
10
11         for (int i = 0; i < str.length(); i++) {
12             char ch = str.charAt(i);
13             if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
14                 vowelCounts.put(ch, vowelCounts.getOrDefault(ch, 0) + 1);
15
16                 // Update most frequent vowel if current one has a higher count
17                 if (vowelCounts.get(ch) > maxCount) {
18                     maxCount = vowelCounts.get(ch);
19                     mostFrequentVowel = ch;
20                 }
21             }
22         }
23
24         return mostFrequentVowel;
25     }
26
27     public static void main(String[] args) {
28         String str = "xayuaba";
29         char result = MostFrequentVowel(str);
30         System.out.println("Most Frequent vowel is: "+result); // Output: a
31     }
32 }

```

64) Fibonacci series

```

1 import java.util.Scanner;
2
3 public class fibonacciSeries {
4
5     // Function to print the Fibonacci series up to n terms
6     public static void printFibonacciSeries(int n) {
7         int num1 = 0, num2 = 1;
8
9         System.out.print("Fibonacci Series: " + num1 + ", " + num2);
10
11        for (int i = 2; i < n; i++) {
12            int num3 = num1 + num2;
13            System.out.print(", " + num3);
14            num1 = num2;
15            num2 = num3;
16        }
17        System.out.println();
18    }
19
20    public static void main(String[] args) {
21        Scanner sc = new Scanner(System.in);
22
23        // Input the number of terms
24        System.out.println("Enter the number of terms:");
25        int n = sc.nextInt();
26
27        // Print the Fibonacci series
28        printFibonacciSeries(n);
29    }
30 }

```

65) Max diff successive elements

Problem Statement

Given an integer array `nums`, return the maximum difference between two successive elements in its sorted form. If the array contains less than two elements return 0;

Input: arr = [3,6,9,1]

Output: 3

```
1 import java.util.*;
2
3 public class maxDiffSuccessiveElements{
4     public static int maxDiff(int arr[], int n){
5         if(n < 2){
6             return 0;
7         }
8         Arrays.sort(arr);
9         int maxdiff = Integer.MIN_VALUE;
10        for(int i = 1; i < n; i++){
11            maxdiff = Math.max(maxdiff, (arr[i]-arr[i-1]));
12        }
13        return maxdiff;
14    }
15
16    public static void main(String[] args) {
17        Scanner sc = new Scanner(System.in);
18
19        System.out.println("Enter size of arr: ");
20        int n = sc.nextInt();
21
22        System.out.print("Enter elements : ");
23        int arr[] = new int[n];
24
25        for(int i = 0; i < n; i++){
26            arr[i] = sc.nextInt();
27        }
28
29        System.out.println("Output: "+ maxDiff(arr, n));
30    }
31 }
```

66) nth Fibonacci

Problem Statement

Given input as n, find nth fibonacci series number.

Fibonacci series example is: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...

Input: 9

Output: 34 (At 9th index number is 34)

```
1 import java.util.*;
2
3 public class nthFibonacci{
4     public static int nthFib(int n){
5         if(n == 0 || n == 1){
6             return n;
7         }
8
9         int a = 0, b = 1;
10        int c = 0;
11        for(int i = 2; i <= n; i++){
12            c = a + b;
13            a = b;
14            b = c;
15        }
16        return c;
17    }
18
19    public static void main(String[] args) {
20        Scanner sc = new Scanner(System.in);
21
22        System.out.println("Enter the index to find number of fibonacci series: ");
23        int n = sc.nextInt();
24
25        System.out.println("Output: " + nthFib(n));
26    }
27 }
```

67) Remove Duplicate

Given an array of integers, write a function to remove duplicate elements.

Input: [1, 2, 2, 3, 4, 4, 5]

Output: [1, 2, 3, 4, 5]

```
1 def RemoveDuplicate(arr):
2     uniqueElem = []
3
4     for elem in arr:
5         if elem not in uniqueElem:
6             uniqueElem.append(elem)
7
8     return uniqueElem
9
10
11 def main():
12     arr = [1, 2, 2, 3, 4, 4, 5]
13     print("Unique elements are: ", RemoveDuplicate(arr))
14
15
16 if __name__ == "__main__":
17     main()
```

68) rhyme words

```
1 def rhymeWords(S, D):
2     best_word = "No Word"
3     max_match_length = 0
4
5     for word in D:
6         if word == S:
7             continue
8
9         # Find the longest matching suffix
10        min_length = min(len(S), len(word))
11        match_length = 0
12
13        for i in range(1, min_length + 1):
14            if S[-i] == word[-i]:
15                match_length += 1
16            else:
17                break
18
19        # Update best_word if this word has a longer matching suffix
20        if match_length > max_match_length:
21            max_match_length = match_length
22            best_word = word
23
24    return best_word
25
26
27 def main():
28     S = "thunder"
29     D = ["pukle", "thunder", "powder", "blender", "under"]
30     output = rhymeWords(S, D)
31     print(output)
32
33
34 if __name__ == "__main__":
35     main()
```